1 Constraint Model

We model the employee scheduling problem using the Google OR-Tools classical CP solver (`pywrapcp`). The model uses two primary decision variables. For each employee e and day d, we define a shift variable `shiftOfEmployeeDay`$[e][d]$ indicating which shift the employee works that day, where shift 0 represents an off day. We also define a duration variable `durationOfEmployeeDay`$[e][d]$ representing the number of hours worked on that day.

The model enforces several classes of constraints. First, shift and duration assignments are linked: if an employee is assigned the off shift then their duration must be zero, while working shifts require the duration to lie between `minConsecutiveWork` and `maxDailyWork`. Second, staffing constraints ensure that each day satisfies minimum demand for each shift and that the total scheduled hours meet the daily operational requirement.

Employee-side constraints regulate work patterns. Weekly workload is bounded between `minWeeklyWork` and `maxWeeklyWork`. During the training period, each employee must experience each shift exactly once, implemented with an `AllDifferent` constraint on the first `numShifts` days. Finally, night-shift assignments are restricted by both a cap on total night shifts and a limit on consecutive night shifts.

Together, these constraints ensure that schedules satisfy all operational requirements while respecting employee workload limits.

2 Schedule Quality

A useful way to evaluate schedule quality is to separate the interests of the business from the interests of the employees. These are often not in alignment; owners care about coverage and labor cost, while employees care more about predictability and fairness. A schedule that is efficient from the perspective of capital is not necessarily equitable from the perspective of labor Lambert (2008). **Our scheduler focuses on the abstract problem, and so does not take the following into consideration.** We examine some of our produced schedules in light of these criteria, but the randomization in our solution may lead to some amount of variance.

From a **business owner's perspective**, desirable schedules:

- Maintain backup capacity, so that unexpected absences can be absorbed without disrupting operations. Our scheduler does not enforce this. For example, the solution to `14_15.sched` contains 2 days with zero backup capacity.
- Keep as many employees as possible below full-time hours, since this can help avoid paying full-time benefits and other associated costs. Our model does not consider this, but tends to assign under 40 hour weeks often since employee time is capped at 40 hours. For example, the solution to `21_30.sched` provisions 90 employee-weeks below 40 hours.

From an **employee's perspective**, desirable schedules:

- Avoid temporally back-to-back schedules, such as an evening shift on day N followed by the first shift (i.e., night) on day $N + 1$. Our scheduler does not enforce this. For example, its

solution to `28_65.sched`, involves 37 such evening→night transitions.

- Distribute undesirable (i.e., night and evening shifts) shifts fairly. Our scheduler enforces this only during the training phase. For instance, in its solution to `21_30.sched`, Employee 6 receives 12 night/evening shifts, while Employee 11 receives only 3.
- Provide predictable hours and avoid too-short shifts, since short shifts can impose disproportionate commuting and scheduling costs on workers. Our model only partially addresses this by enforcing a floor. In `7_17.sched`, 85 of 89 shifts are assigned at the minimum length.

From the perspective of a store manager, these schedules seem more like usable first drafts than final schedules. The generated schedules satisfy all modeled constraints, as verified by the absence of malformed assignments, night-shift violations, or training-phase conflicts. However, several quality issues remain. Many schedules operate with little or no backup capacity; for example, in instance `14_14.sched` there are 26 shift-days with zero slack above the minimum demand. In addition, shifts are often assigned at the minimum allowable duration, with over 90% of shifts at the minimum length in `14_15.sched`. Undesirable shifts are also unevenly distributed, with employees receiving between 3 and 12 such assignments in instance `21_30.sched`. Finally, some schedules include undesirable shift transitions and short rest intervals, such as the 37 evening-to-night transitions observed in `28_65.sched`. These results suggest that while the schedules are feasible, additional constraints or objectives would be required to improve their practical quality.

3 Approach

Some observations guided our search design.

1. Assignments of night shifts are subject to the most constraints, since they are limited by a cap on their total number and on consecutive night shifts. This suggested that, when possible, night shifts should be deferred to less restrictive assignments.
2. For duration variables, it may be helpful to greedily try larger values first, since this more quickly satisfied daily operation and weekly workload constraints.
3. Shift assignments and work durations should be guided together rather than treated as mostly separate parts of the problem.

As a result, we started by implementing a custom "interleaving" search strategy over both types of decision variables (shift and duration). While it reduced the number of conflicts compared to what we had earlier, it performed poorly. We think this is because the implementation was much slower than to OR-Tools' built-in C++ search machinery.

Thus, we moved to an impact-based `DefaultPhase` with Luby restarts and a fallback heuristic. The hope is that this allows the solver to respond to changes in the shift and duration constraints together while avoiding the performance cost of a Python-level custom search. We also used the auxiliary heuristic hook provided by OR-Tools to encode some of our insights through max-value selection, favoring larger duration assignments, and, under our shift encoding, preferring day/evening shifts before night/off when the fallback heuristic is used.

References

Susan J. Lambert. Passing the buck: Labor flexibility practices that transfer risk onto hourly workers. *Human Relations*, 61(9):1203–1227, 2008. doi: 10.1177/0018726708094910. URL https://doi.org/10.1177/0018726708094910.